



University of Global Village (UGV), Barishal

Department of Computer Science and Engineering

Encapsulation

Binding Data and Behavior Together in Java



Course Teacher
MD. MEHADI HASAN
Lecturer

Department of CSE
University of Global Village(UGV), Barishal

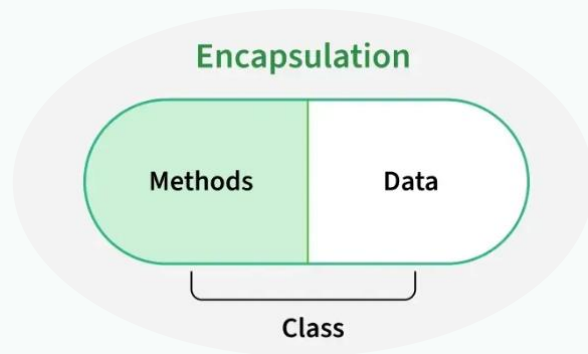
Introduction to Encapsulation

Definition

Encapsulation is the process of wrapping data (variables) and the methods that act on that data into a single unit — a class. It also means restricting direct access to some of an object's internal details.

What is Encapsulation in OOP?

- One of the four pillars of OOP, along with Abstraction, Inheritance, and Polymorphism
- Combines data and the code that operates on it inside one class
- Hides the internal working of an object from the outside world



Key Characteristics

1

Bundling

Groups variables and methods together inside one class.

2

Hiding

Keeps an object's internal state hidden from outside code.

3

Exposure

Reveals only the functionality that is needed, nothing more.

4

Controlled Access

Outside code can reach the data only through defined methods.

Real-World Analogies



Medicine Capsule

A capsule hides the medicine powder inside a shell. You take it as a whole, without touching the ingredients directly.



ATM Machine

You press buttons on the screen to withdraw money. The banking software and account logic stay completely hidden.



Mobile Phone

You use the screen and buttons to make a call. The internal circuits and processors stay sealed inside the case.



Bank Account

Your balance is private. You can deposit or withdraw only through official operations, never by editing it directly.

Why Encapsulation is Important

Encapsulation protects an object's data and keeps a program easier to build, change, and trust over time. It is one of the main reasons OOP code stays organized as projects grow larger.

Why We Use It

- To stop outside code from changing data in unexpected ways
- To keep the internal design of a class free to change later
- To present a clean, simple interface to other programmers

1

Data Hiding

Sensitive fields are kept private and cannot be touched directly.

2

Better Security

Only approved methods can read or change important data.

3

Improved Maintainability

Internal code can change without breaking other parts of the program.

4

Controlled Access

Setters can validate data before it is ever stored.

5

Code Reusability

Well-encapsulated classes are easy to reuse in new projects.

Encapsulation in Java

Role of Classes and Objects

- A class is the blueprint that declares private fields and public methods
- An object is a real instance of the class, created in memory
- The object's private data can be reached only through its own methods

Java gives Encapsulation direct support through access modifiers, especially the private keyword, combined with public getter and setter methods.

Steps to Create an Encapsulated Class

- 1 Declare fields as private**
Mark every instance variable with the private keyword.
- 2 Write public getters**
Add a method to safely return each field's value.
- 3 Write public setters**
Add a method to safely update each field's value.
- 4 Add validation (optional)**
Check values inside setters before storing them.

The private Keyword

Definition

private is an access modifier in Java. A member marked private can be used only inside the class where it is declared.

Syntax

```
private dataType variableName;
```

```
private String name;
```

```
private int age;
```

Why private is Essential for Encapsulation

- It blocks direct, uncontrolled access to a class's data
- It forces all access to go through getter and setter methods
- It is the most restrictive access level in Java

private vs public

private	public
Accessible only inside its own class	Accessible from anywhere in the program
Used to hide sensitive data	Used to expose safe functionality
Cannot be inherited by name directly	Freely inherited and reused
Ideal for instance variables	Ideal for getters, setters, and APIs

Getter and Setter Methods

Definition & Purpose

- A getter is a public method that returns the value of a private field
- A setter is a public method that updates the value of a private field
- Together, they give controlled, safe access to an object's data
- A setter can validate a value before it is ever stored

Naming Convention

- Getter: get + FieldName(), e.g. getName()
- Setter: set + FieldName(value), e.g. setName(value)
- Field names follow camelCase, methods follow the same style

Syntax

```
// Getter
public String getName() {
    return name;
}

// Setter
public void setName(String newName) {
    name = newName;
}
```

Getters and setters act as a controlled “door” to private data — outside code must pass through them.

Syntax of an Encapsulated Class

```
public class ClassName {  
  
    // Step 1: private fields (data hiding)  
    private dataType fieldName;  
  
    // Step 2: public getter  
    public dataType getFieldName() {  
        return fieldName;  
    }  
  
    // Step 3: public setter  
    public void setFieldName(dataType value) {  
        fieldName = value;  
    }  
}
```

Explanation of Each Component

1

class ClassName

Declares the blueprint that will hold data and behavior.

2

private fieldName

Hides the field so it cannot be touched from outside.

3

public getter

Lets outside code safely read the field's value.

4

public setter

Lets outside code safely update the field's value.

Java Example: Student Class

```
public class Student {  
    private String name; // private field - hidden from outside  
    private int age;  
  
    // Getter for name  
    public String getName() {  
        return this.name;  
    }  
  
    // Setter for name  
    public void setName(String name) {  
        this.name = name; // this.name = field, name = parameter  
    }  
  
    // Getter for age  
    public int getAge() {  
        return this.age;  
    }  
  
    // Setter for age  
    public void setAge(int age) {  
        this.age = age;  
    }  
}
```

What This Class Does

- name and age are declared private - hidden from outside code
- getName() and getAge() safely return the current values
- setName() lets outside code update the name
- setAge() checks that the value is greater than 0 before storing it
- This if-check inside the setter is a simple form of data validation

Student Class — Using the Object

Main Method

```
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.setName("Rafi");  
        s1.setAge(20);  
  
        System.out.println("Name: " + s1.getName());  
        System.out.println("Age: " + s1.getAge());  
    }  
}
```

Expected Output

```
Name: Rafi  
Age: 20
```

Line-by-Line Notes

- `new Student()` creates an object of the class in memory
- `s1.setName(...)` is the only way to change the private name field
- `s1.setAge(20)` passes the setter's validation check, so age is stored
- `s1.getName()` and `s1.getAge()` read the values back safely
- The private fields are never accessed directly, only through methods

Practical Example: BankAccount Class

```
public class BankAccount {  
    private double balance;  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance = balance + amount;  
        }  
    }  
  
    public void withdraw(double amount) {  
        if (amount > 0 && amount <= balance) {  
            balance = balance - amount;  
        }  
    }  
}
```

How Encapsulation Protects the Data

- balance is private, so no outside code can set it directly
- deposit() only allows positive amounts to be added
- withdraw() blocks withdrawals larger than the current balance
- These checks stop the account from ever holding an invalid value

Sample Output

```
deposit(500) → balance = 500.0  
withdraw(200) → balance = 300.0
```

Advantages and Disadvantages

Advantages

- Protects data from accidental or unauthorized changes
- Hides complex implementation details from the user
- Makes code easier to test, debug, and maintain
- Allows internal logic to change without affecting other code
- Increases flexibility through validation inside setters
- Supports code reusability across projects

Disadvantages

- Requires writing extra getter and setter methods
- Can make very simple classes feel more verbose
- Slight performance overhead from method calls versus direct access
- Overusing getters/setters without validation can defeat the purpose

Key Takeaways

- 1 Encapsulation wraps data and methods together inside a class
- 2 It hides internal details and protects data from outside interference
- 3 `private` restricts direct access; `public` getters and setters control it
- 4 Getters return values; setters update values, optionally with validation
- 5 Encapsulation improves security, maintainability, and code reusability
- 6 The `Student` and `BankAccount` examples show real, working encapsulated classes

Practice Questions

Theory Questions

- Explain Encapsulation and discuss its advantages with examples. (Understand)
- Explain the role of the private keyword in implementing Encapsulation, and compare it with public. (Understand)
- Identify two common mistakes beginners make with Encapsulation and how to fix them. (Analyze)

Programming Questions

- Write a Java program implementing Encapsulation using a Student class. (Apply)
- Create an Employee class with private fields and getter/setter methods. (Apply)

Reflection Prompt

Before attempting the programming questions:

1. List every field the class needs, and mark each one private.
2. Decide which fields need a setter with validation (e.g. salary must be positive, age must be reasonable).
3. Write a small main method that creates an object and tests every getter and setter.
4. Predict the output on paper before running the program.

The background is a dark teal color. There are two large, lighter teal circles: one in the top right corner and one in the bottom left corner.

Thank You!!